공학사학위논문

차량 경로 최적화를 위한 확산 모델의 강화 학습 방법론

Reinforcement Learning Methodology for Diffusion Models in Vehicle Routing Problems

지도교수 이재욱

2026 년 6 월

서울대학교 공과대학

산업공학과

최 장 혁

초록

본 논문에서는 조합 최적화 문제를 확산 기반 신경망 해법으로 다룬다. 확산 모델이 생성하는 heatmap을 문제 제약을 만족하는 해로 디코딩하는 과정이 성능의 핵심 병목이며, 이를 완화하기 위해 CADO 스타일의 디코더 인식 강화학습 미세조정을 적용한다. TSP, CVRP, MDVRP에 대한 실험을 통해 제안 방법의 성능 및 주요 결과를 보고한다.

주요어: 조합최적화, 확산모델, 강화학습, 신경망 조합최적화, 차량 경로 문제

목차

표 목차

그림 목차

제 1 장 Introduction

1.1 Motivation

Combinatorial optimization (CO) is a fundamental field in computer science and operations research, encompassing the search for an optimal solution from a finite but often exponentially large set of candidates. Classic examples include the traveling salesman problem (TSP), the maximum independent set (MIS), and the vehicle routing problem (VRP). These problems are NP-hard, meaning they are believed to be intractable in polynomial time, and no known algorithm can solve them efficiently at scale.

Traditionally, CO problems have been handled by exact solvers (e.g., Concorde and Gurobi) [1, 4], approximation algorithms, and hand-crafted heuristics such as LKH and local search methods like 2-opt [3, 5, 8]. However, these methods have significant drawbacks, often demanding substantial domain expertise, careful engineering, or considerable computational resources. These limitations have motivated the search for data-driven alternatives. In recent years, neural combinatorial optimization (NCO) has emerged as a promising approach to address the constraints of traditional methods [2].

As the field of NCO has progressed, various architectures have been proposed. In this paper, we utilize a diffusion-based architecture modeled with a Bernouill distribution to solve CO problems for two primary reasons: 1) Diffusion-based architectures can capture high-dimensional, multi-modal distributions, evaluating solution probabilities simultaneously. Due to the inherent complexity of CO problems, this multi-modality is crucial for capturing the underlying problem structure. 2) The diffusion process iteratively refines its output, a mechanism that conceptually mirrors the iterative improvement approaches of classical solvers. Consequently, this research investigates methods to enhance the capabilities of diffusion-based models in solving CO problems.

1.2 Problem Statement

Diffusion-based models generate outputs in the form of a heatmap, representing probability distributions over the solution space. Specifically, discrete diffusion models exploit a Bernoulli distribution. To yield a final, valid solution, this heatmap must be decoded subject to hard problem constraints. However, this decoding process is a primary bottleneck that degrades model performance, as the models are traditionally oblivious to the non-differentiable decoding mechanism and the problem's strict constraints [16]. In CADO, the authors propose a decoder-aware reinforcement learning approach to align heatmap generation explicitly with the decoding process.

Although CADO successfully introduces decoding awareness to heatmap-based models, its application is restricted to the TSP and MIS, which feature relatively simpler structural constraints compared to other CO problems. In this paper, we focus on capacitated vehicle routing problems (CVRPs) and multi-depot vehicle routing problems (MDVRPs). These variants possess significantly more complex constraints within their structure and represent critical challenges in real-world logistics and supply chain management.

1.3 Contributions

- Reproduce and extend *DifusCO*-style diffusion solvers for TSP and CVRP.
- Implement *CADO*-style hybrid fine-tuning and REINFORCE on top of supervised checkpoints.
- Empirical comparison: optimality gap, wall-clock cost, and ablations (reward mode, entropy, denoising steps).

1.4 Thesis Outline

제 2 장 Literature Review

2.1 Optimization Problems

2.1.1 Combinatorial Optimization

This paper follows the conventional notation for combinatorial optimization introduced by Papadimitriou and Steiglitz [10]. The space of candidate solutions for a CO problem instance s is denoted by $\mathcal{X}_s = \{0,1\}^N$, and the objective function is denoted by $c_s : \mathcal{X}_s \to \mathbb{R}$ for a solution $x \in \mathcal{X}_s$. Given a cost function $c_s(\cdot)$, the goal is to find the optimal solution x^{s*} for the given instance s:

$$x^{s*} = \arg\min_{x \in \mathcal{X}_s} c_s(x).$$

Because enumerating all possible solution sets for a CO problem is computationally intractable, exact methods often relax the binary condition, solve a linear program (LP), and subsequently branch on fractional solution components. This approach is known as branch and bound (B&B). While B&B avoids enumerating entirely unnecessary solution sets, the number of branches can still grow exponentially in the worst case.

2.1.2 Traveling Salesman Problem

Let $G = (V, E)$ be a complete undirected graph with $|V| = n$ cities and edge costs $c_{ij} \geq 0$ for all $\{i, j\} \in E$. Define binary decision variables

$$x_{ij} = \begin{cases} 1, & \text{if the tour uses edge } \{i,j\}, \\ 0, & \text{otherwise.} \end{cases}$$

The traveling salesman problem (TSP) can be formulated as the following integer program:

$$
\begin{aligned}
\min_{x} \quad & \sum_{\{i,j\}\in E} c_{ij}x_{ij} \\
\text{s.t.} \quad & \sum_{j\in V\setminus\{i\}} x_{ij} = 2, && \forall i \in V \quad \text{(degree constraints)} \\
& \sum_{\{i,j\}\in E(S)} x_{ij} \le |S| - 1, && \forall S \subset V,\ 2 \le |S| \le n-1 \quad \text{(subtour elimination)} \\
& x_{ij} \in \{0,1\}, && \forall \{i,j\} \in E.
\end{aligned}
$$

Here, $E(S) = \{\{i,j\} \in E : i \in S,\ j \in S\}$ denotes the set of edges with both endpoints in subset S. The subtour elimination constraints enforce that the selected edges form a single Hamiltonian cycle visiting every city exactly once, rather than multiple disjoint cycles.

2.1.3 Capacitated Vehicle Routing Problem

Let $G = (V, E)$ be a complete graph where $V = \{0, 1, \ldots, n\}$, with depot 0 and customers $1, \ldots, n$. Each customer $i \in \{1, \ldots, n\}$ has demand $q_i > 0$, and $c_{ij} \ge 0$ is the travel cost between nodes i and j. A fleet of at most m identical vehicles is available, each with capacity Q. Define binary decision variables

$$
x_{ij} = \begin{cases} 1, & \text{if a vehicle travels directly from } i \text{ to } j, \\ 0, & \text{otherwise,} \end{cases} \qquad \forall i \neq j,\ i,j \in V,
$$

and continuous load variables u_i for $i \in \{1, \ldots, n\}$ denoting the cumulative load delivered upon arrival at customer i.

The capacitated vehicle routing problem (CVRP) can be formulated as the following mixed-

integer program:

$$
\begin{aligned}
\min_{x,u} \quad & \sum_{i \in V} \sum_{j \in V \setminus \{i\}} c_{ij} x_{ij} \\
\text{s.t.} \quad & \sum_{j \in V \setminus \{i\}} x_{ij} = 1, && \forall i \in V \setminus \{0\} \quad \text{(each customer departs once)} \\
& \sum_{i \in V \setminus \{j\}} x_{ij} = 1, && \forall j \in V \setminus \{0\} \quad \text{(each customer arrives once)} \\
& \sum_{j \in V \setminus \{0\}} x_{0j} \leq m, && \text{(number of routes)} \\
& \sum_{i \in V \setminus \{0\}} x_{i0} = \sum_{j \in V \setminus \{0\}} x_{0j}, && \text{(flow conservation at depot)} \\
& q_i \leq u_i \leq Q, && \forall i \in V \setminus \{0\} \quad \text{(capacity bounds)} \\
& u_i - u_j + Q x_{ij} \leq Q - q_j, && \forall i, j \in V \setminus \{0\},\ i \neq j \quad \text{(MTZ)} \\
& x_{ij} \in \{0, 1\}, && \forall i \neq j,\ i, j \in V.
\end{aligned}
$$

The Miller–Tucker–Zemlin (MTZ) constraints enforce route connectivity and eliminate subtours while simultaneously ensuring that vehicle capacity is not exceeded.

2.1.4 Multi-Depot Vehicle Routing Problem

Let $G = (V, E)$ be a complete graph containing multiple depots and customers. Let $D = \{0, 1, \ldots, d\}$ be the set of depots and $C = \{d+1, \ldots, d+n\}$ the set of customers, such that $V = D \cup C$ and $D \cap C = \emptyset$. Depot $k \in D$ has at most m_k identical vehicles available, and each customer $i \in C$ has demand $q_i > 0$. Let $c_{ij} \geq 0$ denote the travel cost between nodes i and j, and let $V_k = C \cup \{k\}$ be the customer set together with depot k. Define depot-indexed binary decision variables

$$
x_{ij}^k = \begin{cases} 1, & \text{if a vehicle from depot } k \text{ travels directly from } i \text{ to } j, \\ 0, & \text{otherwise,} \end{cases} \qquad \forall k \in D,\ i \neq j,\ i, j \in V_k,
$$

and assignment variables

$$y_{ik} = \begin{cases} 1, & \text{if customer } i \text{ is served by depot } k, \\ 0, & \text{otherwise,} \end{cases} \qquad \forall i \in C,\ k \in D.$$

The multi-depot vehicle routing problem (MDVRP) can be formulated as the following mixed-integer program:

$$\begin{aligned}
\min_{x,y} \quad & \sum_{k\in D} \sum_{i\in V_k} \sum_{j\in V_k\setminus\{i\}} c_{ij} x_{ij}^k \\
\text{s.t.} \quad & \sum_{k\in D} y_{ik} = 1, && \forall i \in C \quad \text{(assign each customer to one depot)} \\
& \sum_{j\in V_k\setminus\{i\}} x_{ij}^k = y_{ik}, && \forall i \in C,\ \forall k \in D \quad \text{(customer departure)} \\
& \sum_{j\in V_k\setminus\{i\}} x_{ji}^k = y_{ik}, && \forall i \in C,\ \forall k \in D \quad \text{(customer arrival)} \\
& \sum_{j\in C} x_{kj}^k = \sum_{j\in C} x_{jk}^k \le m_k, && \forall k \in D \quad \text{(depot flow and route count)} \\
& \sum_{i\in S} \sum_{j\in S\setminus\{i\}} x_{ij}^k \le |S| - 1, && \forall k \in D,\ \forall S \subseteq C,\ 2 \le |S| \le n-1 \quad \text{(subtour elimination)} \\
& x_{ij}^k \in \{0,1\}, && \forall k \in D,\ i \ne j,\ i,j \in V_k, \\
& y_{ik} \in \{0,1\}, && \forall i \in C,\ k \in D.
\end{aligned}$$

If vehicle capacities are included, standard capacity constraints, such as MTZ- or flow-based constraints within each depot's routes, can be added to ensure that the total demand on any route does not exceed the vehicle capacity.

2.2 Classical and Learning-Based Solvers

Combinatorial optimization solvers can be broadly grouped into exact methods, relaxation-based methods, and heuristic or metaheuristic methods. In practice, modern state-of-the-art solvers frequently combine these ideas.

2.2.1 Exact Solvers

Exact solvers provide guaranteed optimality, but their computational cost can grow exponentially. Branch and bound (B&B) builds a search tree over partial solutions. At each node, it computes a lower bound, typically through a relaxation such as an LP relaxation. If the bound is worse than the best-known feasible solution, which acts as an upper bound, the node is pruned. B&B is exact, but the search tree can still become exponentially large.

Branch and cut (B&C) augments B&B with cutting planes. When the LP relaxation yields a fractional solution, the solver adds valid inequalities, or cuts, that remove the fractional solution while preserving all integer-feasible solutions. For the TSP, subtour elimination constraints serve as classic cuts. This strategy forms the backbone of solvers such as Concorde [1].

Branch and price (B&P) combines B&B with column generation. Instead of enumerating all variables, which is often exponential, the solver optimizes a restricted master problem and repeatedly generates new variables by solving a pricing subproblem. VRP formulations that use route variables commonly employ this approach.

2.2.2 Relaxation and Decomposition Methods

Relaxation and decomposition methods are widely used to obtain bounds and exploit scalable problem structure. LP relaxation replaces integrality constraints with continuous constraints such as $x \in [0, 1]$, yielding a linear program. LP relaxations provide lower bounds and are standard components in B&B and B&C frameworks, although their fractional solutions may differ substantially from integer-feasible solutions.

Lagrangian relaxation moves hard constraints into the objective function using multipliers. This transformation can produce decomposable subproblems and often yields strong bounds, especially for routing and scheduling problems. Benders decomposition instead splits variables into a master problem and a subproblem. The subproblem generates Benders cuts that iteratively refine the master problem, making the approach effective for problems with naturally separable structure.

2.2.3 Heuristics and Metaheuristics

Heuristics and metaheuristics are designed to obtain high-quality feasible solutions quickly. Local search iteratively improves a feasible solution through small neighborhood moves. In routing problems, common operations include 2-opt, 3-opt, relocate, swap, and cross-exchange. Although local search is fast and effective, it is susceptible to local minima.

Metaheuristics are higher-level strategies designed to escape local minima or explore the solution space more broadly. Prominent examples include simulated annealing (SA), which occasionally accepts worse moves according to a temperature schedule; tabu search, which uses memory to avoid cycling back to recently visited solutions; genetic algorithms (GA), which maintain a population of solutions through crossover and mutation; and ant colony optimization (ACO), which constructs solutions probabilistically based on pheromone trails. These methods do not guarantee optimality but are highly competitive for large instances.

2.2.4 Learning-Assisted Solvers

Recent neural CO research often integrates traditional solvers in two distinct roles. First, exact or high-quality heuristic solutions can serve as teachers or oracles, providing ground-truth labels for supervised learning. Second, classical methods can act as decoding or refinement steps: neural models generate a soft solution, such as a probabilistic heatmap, which is subsequently transformed into a feasible hard solution using greedy decoding, local search such as 2-opt, or other solver components.

In summary, exact solvers provide optimality guarantees at high computational cost, relaxations and decompositions offer scalable bounds, and heuristics provide strong solutions rapidly. Modern NCO systems frequently combine all three, aiming to address the limitations of traditional solvers through data-driven advantages.

2.3 Neural Network Methodologies

2.3.1 Graph Neural Networks

Graph neural networks (GNNs) are neural architectures designed to process data defined on graphs by repeatedly performing message passing. Each node updates its representation by aggregating transformed features from its neighbors. In each layer, a generic GNN computes

$$h_i^{\ell+1} = \phi\Big(h_i^{\ell},\ \mathrm{AGG}_{j\in\mathcal{N}(i)}\,\psi(h_i^{\ell}, h_j^{\ell}, e_{ij})\Big),$$

where h_i^{ℓ} is the node embedding at layer ℓ, e_{ij} is an optional edge feature, and AGG is a permutation-invariant operator such as summation or mean aggregation.

In neural combinatorial optimization, classical backbones such as graph convolutional networks (GCNs) [7] and graph attention networks (GATs) [13] primarily produce node embeddings. This is sufficient when the target variable is node-based, such as predicting whether a node is selected in the maximum independent set (MIS). However, for routing problems such as the TSP, the key latent variable is often edge-based, indicating whether an edge belongs to the tour. Consequently, a standard node-centric GNN requires additional structural mechanisms to reliably represent and predict edge decisions.

2.3.2 Anisotropic Graph Neural Networks

Anisotropic graph neural networks (AGNNs) extend traditional message passing by maintaining both node and edge embeddings and by using edge gates so that information flow depends on the current edge state. This architecture is especially useful for diffusion-style denoising networks, where the model ingests noisy variables x_t, such as node states for MIS or edge states for TSP, and predicts the clean variables x_0.

Let h_i^{ℓ} and e_{ij}^{ℓ} denote node and edge features at layer ℓ, and let t be a sinusoidal timestep embedding. A standard AGNN layer used in diffusion backbones first performs an anisotropic edge update:

$$\hat{e}_{ij}^{\ell+1} = P^{\ell} e_{ij}^{\ell} + Q^{\ell} h_i^{\ell} + R^{\ell} h_j^{\ell}.$$

The edges are then refined with residual multilayer perceptrons (MLPs) and timestep conditioning:

$$e_{ij}^{\ell+1} = e_{ij}^{\ell} + \mathrm{MLP}_e(\mathrm{BN}(\hat{e}_{ij}^{\ell+1})) + \mathrm{MLP}_t(t).$$

The node update then uses edge-gated messages:

$$h_i^{\ell+1} = h_i^{\ell} + \alpha\Big(\mathrm{BN}\left(U^{\ell}h_i^{\ell} + \sum_{j\in\mathcal{N}(i)} \sigma(\hat{e}_{ij}^{\ell+1}) \odot V^{\ell}h_j^{\ell}\right)\Big).$$

Here $\sigma(\cdot)$ is a sigmoid gate and $\odot$ is the Hadamard product. Intuitively, each neighbor's message is filtered by the current edge embedding, allowing the network to emphasize or suppress interactions in a direction- and edge-dependent manner.

Because AGNNs jointly model nodes and edges, they naturally support optimization tasks that require edge-variable prediction, such as the TSP. For these problems, e_{ij}^0 is initialized from the noisy edge variables x_t, and h_i^0 is set to positional or sinusoidal node features. The final network head then predicts denoised edge probabilities or values.

2.4 Diffusion Models for COP

2.4.1 DIFUSCO

DIFUSCO (Diffusion for Combinatorial Optimization) solves CO problems by casting a feasible solution as a binary structure, learning a diffusion-style denoising process to transform random noise into that structure, and finally applying a classical decoder, such as greedy decoding followed by 2-opt, to yield a valid tour or route [12].

Representation

DIFUSCO represents the target solution as a binary variable $x_0 \in \{0,1\}^N$. For TSP, x_0 typically corresponds to edge decisions, indicating whether an edge is included in the tour. Thus, N scales with the number of candidate edges.

Forward Process

A Markov chain gradually corrupts the optimal solution x_0 into random noise x_T using a variance schedule $\{\beta_t\}_{t=1}^T$. In discrete diffusion, the solution x is converted to a one-hot representation $\tilde{x} \in \{0,1\}^{N\times 2}$ and subjected to a categorical transition:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}(x_t;\, p = \tilde{x}_{t-1}Q_t), \quad Q_t = \begin{pmatrix} 1-\beta_t & \beta_t \\ \beta_t & 1-\beta_t \end{pmatrix}.$$

As $\prod_{t=1}^T (1-\beta_t) \approx 0$, the terminal distribution becomes near-uniform.

For continuous diffusion on discrete data, the discrete solution $x_0 \in \{0,1\}^N$ is rescaled to $\hat{x}_0 \in \{-1,1\}^N$ and subjected to Gaussian corruption:

$$q(\hat{x}_t \mid \hat{x}_{t-1}) := \mathcal{N}(\hat{x}_t;\, \sqrt{1-\beta_t}\,\hat{x}_{t-1},\, \beta_t I).$$

This ensures that $\hat{x}_T$ approaches a standard Gaussian distribution when $\prod_{t=1}^T (1-\beta_t) \approx 0$.

Reverse Process

DIFUSCO learns the reverse transitions defined by

$$p_\theta(x_0) := \int p_\theta(x_{0:T})\, dx_{1:T}, \quad p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t).$$

This is achieved by optimizing the standard variational lower bound, which yields Kullback–Leibler (KL) divergence terms of the form

$$D_{\mathrm{KL}}\big(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\big).$$

The neural network prediction target depends on the diffusion type. In discrete diffusion, the denoiser is trained to predict the clean variable $p_\theta(x_0 \mid x_t)$. The reverse step is formed by

substituting the prediction into the true posterior:

$$p_\theta(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t, \hat{x}_0), \quad \hat{x}_0 \sim p_\theta(x_0 \mid x_t).$$

In continuous diffusion, the denoiser predicts the Gaussian noise ε_t:

$$\varepsilon_t = \frac{\hat{x}_t - \sqrt{\bar{\alpha}_t}\,\hat{x}_0}{\sqrt{1-\bar{\alpha}_t}} = f_\theta(\hat{x}_t, t),$$

which is used to formulate a point estimate of $\hat{x}_0$ inside the Gaussian posterior for $\hat{x}_{t-1}$.

AGNN Denoiser

The conditional model $f_\theta(\cdot, t)$ is implemented using an edge-aware AGNN backbone with timestep conditioning. After sampling the denoised representation to obtain a continuous or probabilistic x_0, the model applies thresholding or quantization to map the heatmap back to the $\{0, 1\}$ domain. Finally, a non-differentiable decoder, such as greedy edge selection, and optional local improvement heuristics, such as 2-opt, are applied to enforce hard feasibility constraints and minimize the final objective value.

2.5 Reinforcement Learning Fine-Tuning

2.5.1 CADO

While diffusion-based models such as DIFUSCO demonstrate strong performance using supervised learning (SL), this paradigm suffers from a fundamental objective mismatch. In SL, the neural network is trained to imitate optimal ground-truth solutions by minimizing surrogate losses, such as cross-entropy. However, minimizing this imitation loss does not guarantee minimizing the actual decoded solution cost.

CADO (Cost-Aware Diffusion Solvers for Combinatorial Optimization through RL Fine-Tuning) analyzes this objective mismatch through two main deficiencies [16]. The first is decoder-blindness: the SL objective is oblivious to the non-differentiable decoding heuristic f_g, such as

greedy search or 2-opt, that projects the continuous heatmap into a discrete feasible solution. The second is cost-blindness: SL prioritizes structural similarity to the ground-truth solution over the actual objective value of the problem. Empirical results in CADO show that structural proximity to an optimal solution, measured by Hamming distance, can have near-zero correlation with the final decoded solution cost.

To address these issues, CADO introduces a reinforcement learning (RL) fine-tuning framework that explicitly aligns the diffusion process with the true CO objective.

2.5.2 MDP Formulation

Instead of relying only on structural imitation, CADO directly optimizes the post-decoded solution cost by formulating the iterative diffusion denoising process as a Markov decision process (MDP). This formulation enables the model to be trained using the REINFORCE algorithm.

Let the MDP be represented by the tuple $(\mathcal{S}, \mathcal{A}, P, \rho_0, R)$. The state $s_t \in \mathcal{S}$, action $a_t \in \mathcal{A}$, and reward function $R(s_t, a_t)$ at timestep t are defined as

$$s_t \triangleq (g, T-t, x_{T-t}), \quad a_t \triangleq x_{t-1},$$

and

$$R(s_t, a_t) \triangleq \begin{cases} -c_g(f_g(x_0)), & \text{if } t = T, \\ 0, & \text{otherwise.} \end{cases}$$

Here, g represents the problem instance, $c_g(\cdot)$ is the true cost function, and $f_g(\cdot)$ is the non-differentiable decoder. By evaluating the reward only at the final step, after the heatmap has been fully decoded, this formulation explicitly incorporates both the decoder's behavior and the true cost feedback, resolving decoder-blindness and cost-blindness.

2.5.3 Reward Strategies

To stabilize RL training and reduce variance, CADO uses two reward formulation strategies. The standard reward (SR) uses the negative true solution cost directly as the reward signal.

This label-free formulation enables the model to learn through trial-and-error exploration and can remain applicable even when labeled solutions are unavailable.

The label-centered reward (LCR) uses information from the SL pre-training dataset without returning to pure imitation. It repurposes the ground-truth solution cost $b_D(g)$ as an instance-specific baseline and defines the reward as the negative optimality gap:

$$R_t = -\big(c_g(f_g(x_0)) - b_D(g)\big).$$

Because the baseline $b_D(g)$ depends only on the problem instance and is independent of the policy's actions, it serves as an unbiased baseline. Thus, the policy gradient remains consistent with true cost minimization even if the dataset labels are computationally suboptimal.

2.5.4 Hybrid Fine-Tuning

Applying RL fine-tuning naively to the large AGNN architectures used in diffusion models can be unstable and memory-intensive. To achieve parameter-efficient and stable adaptation, CADO employs a hybrid fine-tuning (Hybrid-FT) strategy.

Hybrid-FT applies low-rank adaptation (LoRA) to the input layer and most backbone GNN layers. This preserves the robust features learned during SL pre-training while introducing only a small set of trainable low-rank matrices. At the same time, CADO applies selective layer fine-tuning (Selective-FT), in which all parameters are retrained, to the final GNN layer and the output layer. These layers are most directly responsible for final heatmap generation. The combination supports rapid initial convergence while maintaining sufficient expressive power for cost-aware adaptation.

제 3 장 Methodology

3.1 Overview

This thesis develops and evaluates diffusion-based neural solvers for routing problems. The proposed methodology follows a two-stage training pipeline. First, a DIFUSCO-style graph diffusion model is trained with supervised denoising using solver-generated reference solutions [12]. Second, the supervised checkpoint can be further adapted using a CADO-style reinforcement learning objective that directly rewards the quality of the decoded solution [16].

The motivation for this design is that supervised denoising provides stable training for high-dimensional binary solution structures, while reinforcement learning fine-tuning reduces the mismatch between the training objective and the final evaluation objective. In supervised learning, the model is trained to imitate reference binary labels. However, routing performance is ultimately determined only after the predicted heatmap is passed through a non-differentiable decoder. Therefore, the second stage explicitly incorporates the decoder and the decoded solution cost into the training signal.

The implementation covers three routing settings: the Traveling Salesman Problem (TSP), the Capacitated Vehicle Routing Problem (CVRP), and the Multi-Depot Vehicle Routing Problem (MDVRP). For TSP, the model predicts a heatmap over candidate tour edges. For CVRP, the same edge-heatmap formulation is extended to multi-route vehicle routing by adding depot and demand information to the node representation and by enforcing capacity during decoding. For MDVRP, this thesis considers an assignment-level formulation in which the model predicts customer–depot assignment scores. Therefore, TSP and CVRP are evaluated using decoded route cost and relative gap to a reference solution, whereas MDVRP is evaluated using assignment-level metrics such as depot-assignment accuracy and capacity-violation rate.

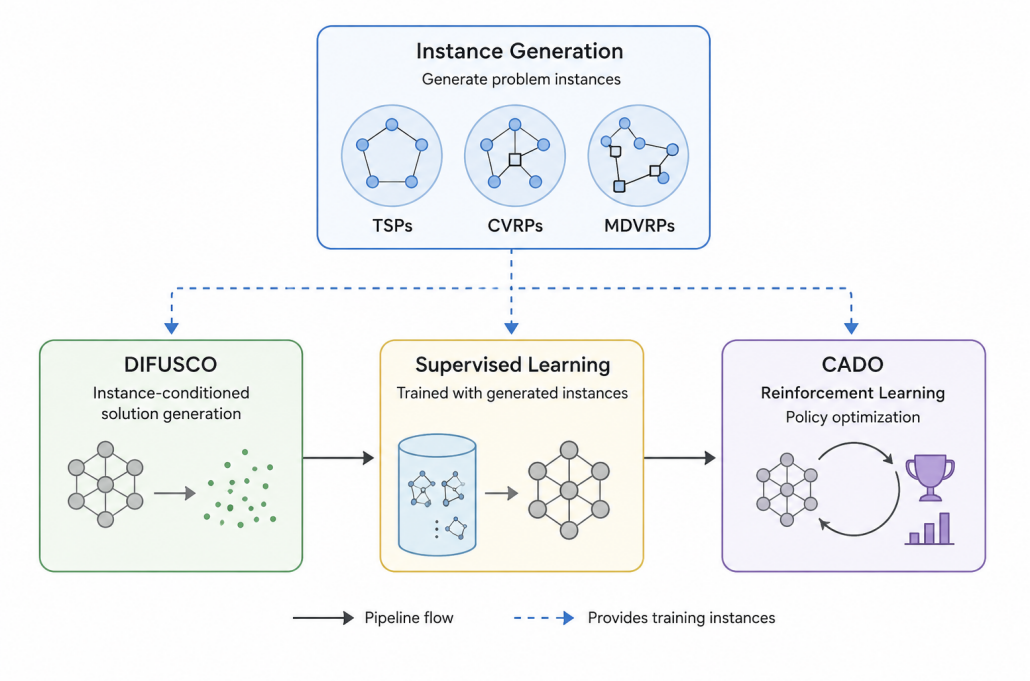

Figure 3.1: Overview of the proposed training and evaluation pipeline. A DIFUSCO-style diffusion model is first trained by supervised denoising on solver-generated binary labels [12]. The trained checkpoint can then be fine-tuned with a CADO-style reinforcement learning objective based on decoded solution quality [16].

3.2 Binary Solution Representation

Following the graph-based diffusion formulation of DIFUSCO [12], each optimization instance is represented by a binary solution vector. Let s denote a problem instance and let

$$x_0 \in \{0,1\}^{|\mathcal{B}_s|}$$

denote the clean binary solution label, where $\mathcal{B}_s$ is the set of candidate binary decisions for instance s. The meaning of each binary variable depends on the problem setting.

For TSP, $\mathcal{B}_s$ consists of candidate edges between cities. A variable $x_{ij} = 1$ indicates that edge (i, j) is included in the tour. A feasible decoded solution must form a Hamiltonian cycle in which every city has degree two and all cities belong to a single connected tour.

For CVRP, $\mathcal{B}_s$ consists of candidate arcs between the depot and customers. A variable $x_{ij} = 1$ indicates that a vehicle travels directly from node i to node j. A feasible decoded solution must

visit every customer exactly once, start and end each route at the depot, and ensure that the total demand assigned to each vehicle route does not exceed the vehicle capacity Q.

For MDVRP, this thesis uses an assignment-level binary representation. Let D be the set of depots and C be the set of customers. The binary variable

$$x_{di} = \begin{cases} 1, & \text{if customer } i \text{ is assigned to depot } d, \\ 0, & \text{otherwise} \end{cases}$$

represents a customer–depot assignment decision. A feasible assignment should assign every customer to exactly one depot and should not exceed the available aggregate capacity of each depot. This representation captures the higher-level decision in MDVRP, but it does not directly construct the vehicle routes within each depot. After assignment, a full MDVRP solution would still require solving one CVRP subproblem for each depot.

3.3 Discrete Diffusion Training

The supervised training stage follows the discrete diffusion process used for graph-based combinatorial optimization in DIFUSCO [12]. Starting from a clean reference solution x_0, the forward process gradually corrupts the binary variables into noisy states x_t for timesteps $t = 1, \ldots, T$. In the discrete Bernoulli diffusion setting, each binary variable is represented as a two-class categorical variable. The transition matrix at timestep t is

$$Q_t = \begin{pmatrix} 1 - \beta_t & \beta_t \\ \beta_t & 1 - \beta_t \end{pmatrix},$$

where β_t is the corruption probability. Thus, the forward transition is defined as

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\left(x_t; p = \tilde{x}_{t-1} Q_t\right),$$

where $\tilde{x}_{t-1}$ denotes the one-hot representation of x_{t-1}.

During training, a timestep t is sampled, the corresponding noisy solution x_t is generated, and the denoising network predicts the clean binary label x_0. The network receives the problem instance s, the noisy binary state x_t, and the timestep t, and returns two-class logits for each candidate decision:

$$z_\theta = f_\theta(s, x_t, t).$$

The supervised denoising objective is the cross-entropy loss between the predicted logits and the clean reference solution:

$$\mathcal{L}_{\mathrm{SL}}(\theta) = \mathbb{E}_{s,x_0,t,x_t}\left[\frac{1}{|\mathcal{B}_s|}\sum_{b\in\mathcal{B}_s}\mathrm{CE}\left(z_{\theta,b}, x_{0,b}\right)\right].$$

The probability assigned to class 1 is interpreted as the heatmap value:

$$A_b = P_\theta(x_{0,b} = 1 \mid s, x_t, t) = \mathrm{softmax}(z_{\theta,b})_1.$$

This heatmap is later decoded into a feasible or repaired discrete solution.

3.4 Graph Denoising Architecture

The denoising model is implemented as an edge-aware graph neural network following the AGNN-style backbone used in DIFUSCO [12]. The backbone maintains both node embeddings and edge embeddings, which is important because the main prediction target in routing problems is edge-based or assignment-edge-based.

The TSP model receives node coordinates, edge indices, edge distances, the noisy edge state x_t, and the diffusion timestep t. The initial node representation is obtained from a sinusoidal positional embedding of the two-dimensional coordinates:

$$h_i^{(0)} = \phi_{\mathrm{pos}}(p_i),$$

where $p_i = (x_i, y_i)$ is the coordinate of city i. The initial edge representation is the concatenation

of a distance embedding and a noisy-state embedding:

$$e_{ij}^{(0)} = [\phi_{\text{dist}}(d_{ij}); \phi_{\text{noise}}(x_{t,ij})] \,,$$

where d_{ij} is the Euclidean distance between nodes i and j.

For CVRP, the node representation is extended to include demand and depot information. Each node feature is defined as

$$u_i = [x_i, y_i, q_i/Q, \mathbb{I}(i = \text{depot})],$$

where q_i is the demand of node i, Q is the vehicle capacity, and $\mathbb{I}(i = \text{depot})$ indicates whether the node is the depot. This means that capacity information is available to the neural network through the normalized demand feature q_i/Q. However, the supervised objective remains an edge-wise denoising loss and does not explicitly penalize capacity violations. Capacity feasibility is handled by the decoder and evaluation procedure.

For MDVRP, the node representation is

$$u_i = [x_i, y_i, q_i/Q, r_i],$$

where r_i is a role indicator distinguishing depot nodes from customer nodes. The same edge-denoising backbone is used, but the candidate edge set is restricted to customer–depot assignment edges in the MDVRP experiment. Thus, the MDVRP model predicts assignment heatmaps rather than complete route structures.

The timestep t is embedded using a sinusoidal timestep embedding and projected through a small multilayer perceptron:

$$\tau_t = \text{MLP}_t(\text{Embed}(t)).$$

At each AGNN layer, node and edge embeddings are updated jointly:

$$(H^{(\ell+1)}, E^{(\ell+1)}) = \text{AGNNLayer}\left(H^{(\ell)}, E^{(\ell)}, \mathcal{E}, \tau_t\right),$$

where $\mathcal{E}$ is the candidate edge or assignment-edge set. After the final layer, an edge-level classification head maps each final edge embedding to two logits:

$$z_{ij} = \mathrm{MLP}_{\mathrm{edge}}\left(e_{ij}^{(L)}\right).$$

The resulting class-1 probabilities form the heatmap used for decoding.

3.5 Problem-Specific Decoding

The diffusion model produces heatmap scores, but heatmaps do not automatically satisfy the hard constraints of routing problems. Therefore, each problem requires a problem-specific decoder that maps the predicted scores into a feasible or repaired solution.

For TSP, the decoder ranks undirected candidate edges using the sum of the two directed heatmap scores normalized by Euclidean distance, as in heatmap decoding for DIFUSCO [12]. Edges are inserted into the partial solution only if they do not violate degree constraints and do not create a premature subtour. The process continues until a complete Hamiltonian cycle is formed. A local search heuristic such as 2-opt [3] can optionally be applied to improve the decoded tour; in the evaluation code, 2-opt is applied to the decoded TSP tour.

For CVRP, the decoder uses a similar score-ranked edge insertion procedure. Customer nodes are constrained to degree two, customer-only subtours are forbidden, and capacity feasibility is enforced by tracking the demand of each partial customer chain before it is connected to the depot. If some customers remain open after the greedy pass, they are attached to the depot as a repair step. The decoded CVRP cost is the sum of Euclidean distances over all recovered vehicle routes.

For MDVRP, the decoder operates on the customer–depot assignment heatmap. The directed scores for each customer–depot pair are averaged into an assignment score matrix. Customers are then processed in descending order of confidence, measured by the margin between the best depot score and the median depot score. Each customer is assigned to the highest-scoring depot with enough remaining aggregate capacity. If no depot has enough remaining capacity, the customer is

assigned to the depot with the largest remaining capacity and this forced assignment is counted as an over-capacity event. Since this experiment stops at the assignment stage, it does not construct depot-specific vehicle routes. Consequently, MDVRP is evaluated using assignment-level metrics rather than full MDVRP routing cost.

3.6 CADO-Style Reinforcement Learning Fine-Tuning

The supervised denoising objective trains the model to imitate solver-generated reference solutions. However, minimizing cross-entropy against reference labels does not necessarily minimize the cost of the final decoded solution. This mismatch occurs because the supervised loss is not aware of the non-differentiable decoder and does not directly optimize the routing objective. To reduce this mismatch, this thesis applies CADO-style reinforcement learning fine-tuning after supervised pretraining [16].

Let $g_s(\cdot)$ denote the problem-specific decoder and let $C_s(\cdot)$ denote the decoded solution cost. The reverse diffusion process is treated as a Markov decision process. At denoising timestep t, the state contains the instance, the timestep, and the current noisy solution:

$$\sigma_t = (s, t, x_t).$$

The action is the next denoised sample:

$$a_t = x_{t-1}.$$

The policy is the learned reverse transition:

$$\pi_\theta(a_t \mid \sigma_t) = p_\theta(x_{t-1} \mid x_t, s).$$

In implementation, fine-tuning does not roll out all T diffusion steps. Instead, a shorter inference schedule of M_{train} steps is sampled, starting from Bernoulli noise $x_T \sim \text{Bernoulli}(0.5)$. The stochastic posterior transitions before the last scheduled step produce the actions and log

probabilities used for policy-gradient training. At the final scheduled step, the predicted class-1 probabilities are thresholded to obtain the binary heatmap. The heatmap is then decoded into a solution:

$$\hat{x} = g_s(A_\theta).$$

For TSP and CVRP, the reward is defined as the negative decoded route cost:

$$R_{\mathrm{SR}} = -C_s(\hat{x}).$$

This is the standard reward formulation. Lower-cost decoded solutions receive higher rewards. The implemented rewards do not add an explicit violation penalty. Instead, feasibility is mainly handled by the decoders, and remaining violations are reported as evaluation metrics.

When a reference solution cost is available, this thesis also considers a label-centered reward:

$$R_{\mathrm{LCR}} = -\left(C_s(\hat{x}) - C_s(x^{\mathrm{ref}})\right).$$

Here, $C_s(x^{\mathrm{ref}})$ serves as an instance-dependent baseline. Since this baseline depends only on the problem instance and not on the sampled denoising trajectory, it reduces variance without changing the expected policy-gradient direction.

For MDVRP, the current implementation does not use a route-cost reward because the model predicts only customer–depot assignments. The reward is instead the negative assignment miss rate,

$$R_{\mathrm{MDVRP}} = -\left(1 - \mathrm{Acc}_{\mathrm{assign}}\right).$$

In this assignment-level setting, the SR and LCR modes both use the same negative miss-rate reward.

The REINFORCE gradient estimator [14] is estimated as

$$\nabla_\theta J(\theta) = \mathbb{E}\left[A \sum_{m=1}^{M_{\mathrm{train}}-1} \nabla_\theta \bar{\ell}_m\right],$$

where A is the scalar advantage and $\bar{\ell}_m$ is the mean Bernoulli log probability of the sampled transition at scheduled step m. For TSP and CVRP, this mean is taken over all candidate edges. For MDVRP, it is taken only over customer–depot bipartite edges, so that customer–customer context edges do not dilute the assignment-learning signal. In SR mode, the advantage is the batch-standardized reward. In LCR mode for TSP and CVRP, the centered reward R_{LCR} is used directly as the advantage.

The implemented REINFORCE loss is

$$\mathcal{L}_{\mathrm{RL}} = -A \sum_{m=1}^{M_{\mathrm{train}}-1} \bar{\ell}_m - 0.01 \left(-\frac{1}{B} \sum_{i=1}^{B} \sum_{m=1}^{M_{\mathrm{train}}-1} \bar{\ell}_{i,m} \right),$$

where B is the accumulated batch size. The second term is a sampled negative-log-probability bonus used as a practical exploration proxy; it is not a separately computed Shannon entropy of the full reverse-transition distribution.

3.7 Hybrid Fine-Tuning Strategy

Fine-tuning the entire diffusion backbone with reinforcement learning can be unstable and memory-intensive. To improve stability, this thesis uses the supervised checkpoint as the initialization for RL fine-tuning. The fine-tuning stage updates only the parameters selected for adaptation while preserving most of the representation learned during supervised denoising.

In the hybrid fine-tuning setting, all parameters are frozen first. The final AGNN layers selected for adaptation and the edge-classification head are then fully unfrozen. In the earlier AGNN layers, low-rank adapters [6] are inserted into the linear maps P, Q, R, U, and V, while the original base weights remain frozen. For a base linear map W, the adapted computation has the form

$$Wx + \frac{1}{r} W_{\mathrm{up}} W_{\mathrm{down}} x,$$

where r is the LoRA rank. In the experiments, the default configuration uses rank $r = 2$ and fully updates the last graph layer together with the output head. This design reflects the intuition that early graph layers capture reusable geometric and relational features, whereas the final layers

are more responsible for producing the heatmap that interacts with the decoder. The same fine-tuning interface supports ablations over reward type, algorithm choice, and the number of denoising steps.

3.8 Evaluation Metrics

The evaluation protocol depends on the output space of each problem. During evaluation, the model uses deterministic generation with M_{eval} scheduled denoising steps, rather than the stochastic rollout used to collect RL trajectories.

For TSP, the primary metric is the decoded tour length. When a reference solution is available, the relative gap is computed as

$$\text{Gap}(\%) = \frac{C_{\text{model}} - C_{\text{ref}}}{C_{\text{ref}}} \times 100.$$

Here, C_{model} is the cost of the decoded model solution and C_{ref} is the cost of the reference solution. If the reference solution is solver-generated but not proven optimal, this metric is interpreted as a relative gap to the reference solution rather than a true optimality gap.

For CVRP, the decoded cost is the total length of all vehicle routes. The same relative gap formula is used. In addition, feasibility is checked by counting decoded routes whose load exceeds the vehicle capacity Q. The reported over-capacity rate is the average number of over-capacity routes per evaluated instance.

For MDVRP, the model output is a depot-assignment heatmap. Therefore, the evaluation uses assignment-level metrics. Depot-assignment accuracy is defined as

$$\text{Acc}_{\text{assign}} = \frac{1}{|C|} \sum_{i \in C} \mathbb{I}\left(\hat{d}(i) = d^{\text{ref}}(i)\right),$$

where $\hat{d}(i)$ is the predicted depot for customer i, and $d^{\text{ref}}(i)$ is the reference depot assignment. The MDVRP decoder attempts to avoid aggregate depot-capacity violations during assignment. If a customer must be assigned when no depot has enough remaining capacity, the decoder

records a forced over-capacity assignment. The reported capacity-violation rate is

$$\text{ViolRate} = \frac{N_{\text{overcap}}}{|C|},$$

where N_{overcap} is the number of customers that were forced into an over-capacity depot during decoding. For diagnostic analysis, one can also compute depot-level excess demand,

$$\text{Excess} = \frac{1}{|D|} \sum_{d \in D} \frac{\max\left(0, \sum_{i \in C : \hat{d}(i) = d} q_i - m_d Q\right)}{m_d Q}.$$

These metrics reflect the intended scope of the MDVRP experiment: evaluating whether the diffusion model can learn meaningful customer–depot assignments under capacity constraints.

제 4 장 Experiments

4.1 Experimental Setup

The experiments evaluate whether diffusion-based heatmap solvers can be trained and fine-tuned for increasingly constrained routing problems. TSP is used as the baseline setting, CVRP tests whether the approach remains useful under vehicle-capacity constraints, and MDVRP tests an initial multi-depot assignment extension. The supervised solver follows DIFUSCO [12], and the RL fine-tuning stage follows the CADO framework [16].

Two compute environments were used. TSP supervised and CADO experiments were run locally on an Apple M4 Pro machine with 24 GB unified memory using the PyTorch MPS backend. Larger CVRP and MDVRP supervised runs were executed remotely on a VESSL A100 instance. All main runs use seed 42. Supervised training uses AdamW [9] with learning rate 2×10^{-4}, weight decay 10^{-4}, cosine learning-rate decay, and gradient clipping at norm 1.0. CADO fine-tuning uses AdamW with learning rate 10^{-4}, no weight decay, LoRA rank 2 [6], one selectively fine-tuned final layer, and $M_{\text{train}} = 5$. The CADO evaluation step count follows the individual experiment configuration.

4.2 Datasets

The datasets consist of synthetic Euclidean routing instances with solver-generated labels. TSP-50 contains 128,000 instances solved by Concorde [1]. CVRP-50 contains single-depot instances with 50 customers generated from the X-style distribution and solved by PyVRP [15]. The CVRP file used by the main run contains 128,000 labeled instances. The main MDVRP run uses a generated dataset of 64,000 multi-depot instances with 50 customers.

For supervised training, the dataset is split into 90% training and 10% validation. To keep validation cost manageable, each validation pass evaluates a fixed subset: 500 instances for the supervised TSP, CVRP, and MDVRP runs, 200 instances for the TSP CADO run, and 100

Table 4.1: Before/after comparison for CADO-style RL fine-tuning [16]. For TSP and CVRP, Δ is the reduction in optimality gap. For MDVRP, Δ is the gain in assignment accuracy within the CADO fine-tuning run.

Problem	Metric	Before RL	After RL	Δ	Notes
TSP-50	Gap	4.46%	3.77%	0.69 pp lower	1000-epoch CADO
CVRP-50	Gap	25.96%	20.24%	5.72 pp lower	best epoch 470 CADO
MDVRP-50	Assignment acc.	78.02%	80.86%	2.84 pp higher	CADO epoch 10 to 390

instances for the CVRP and MDVRP CADO runs.

4.3 Baselines and Metrics

The primary baseline is the supervised DIFUSCO checkpoint trained with cross-entropy on solver-generated labels [12]. The central comparison in this chapter is the quality before and after CADO-style RL fine-tuning [16]. For TSP and CVRP, the main metric is optimality gap:

$$\text{Gap}(\%) = \frac{L_{\text{pred}} - L_{\text{gt}}}{L_{\text{gt}}} \times 100.$$

For TSP, the decoded length L_{pred} is measured after greedy decoding and 2-opt post-processing [3]. For CVRP, the supervised comparison includes the best greedy+2-opt validation result, while the CADO validation uses the implemented greedy CVRP decoder and reports the capacity-violation rate. For MDVRP, the current implementation reports depot assignment accuracy and capacity-violation rate, because full route-cost evaluation for MDVRP is not yet integrated into the training loop. Therefore, lower values are better for TSP/CVRP gap, while higher values are better for MDVRP assignment accuracy.

4.4 Main Results

The summary in table 4.1 and fig. 4.1 shows the main experimental direction: after RL fine-tuning, the selected CADO checkpoints improve the decoded metric for all three problem settings. The most direct gap reductions are observed on TSP-50 and CVRP-50. The MDVRP result is assignment-level rather than route-cost based; it shows that the CADO run improves

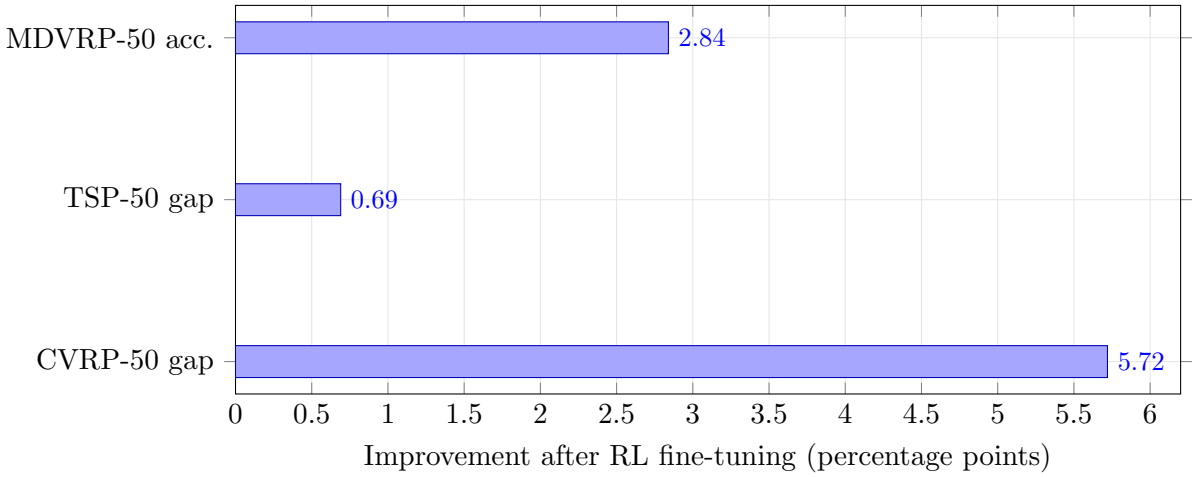

Figure 4.1: Improvement after CADO-style RL fine-tuning [16]. TSP and CVRP report reductions in optimality gap, while MDVRP reports the assignment-accuracy gain from the first CADO validation point to the best CADO checkpoint.

over its own initial validation accuracy and maintains feasibility, although it does not exceed the separately reported best supervised checkpoint.

For TSP-50, the supervised small-model checkpoint reaches a 4.46% validation gap. The 1000-epoch CADO run improves this to 3.77%, a 0.69 percentage-point absolute reduction and a 15.4% relative reduction in gap. This result supports the main conclusion that RL fine-tuning improves the supervised starting point, although the improvement remains modest compared with classical solver quality.

4.5 Ablations

Several implementation-level ablations and comparisons were informative.

Effect of 2-opt on CVRP. The CVRP supervised baseline was evaluated with and without per-route 2-opt [3]. Greedy decoding alone reached approximately 29.98% best validation gap, while enabling 2-opt improved the best gap to 25.96%. This confirms that local refinement is an important part of the heatmap-to-route pipeline for CVRP.

Reward mode. For TSP, LCR was ineffective with the weak supervised baseline used in this thesis: the reward produced little usable relative signal. Switching to SR with batch-normalized

advantages and learning rate 10^{-4} produced measurable improvements. For CVRP, the main CADO configuration uses LCR, but the run still exhibits strong instability, suggesting that reward scaling and checkpoint selection remain central issues.

Training duration. Longer CADO training is not uniformly beneficial. On TSP, the 1000-epoch run improves the average validation band only slowly and remains noisy. On CVRP, the 1000-epoch run improves until the middle of training and then collapses. On MDVRP, 500 epochs are enough to recover from the early dip and finish near the best assignment accuracy. These results support validation-based checkpoint selection rather than selecting the final checkpoint by default.

Parameter-efficient fine-tuning. Hybrid-FT substantially reduces the number of trainable parameters. In the TSP small-model run, only 178K of 953K parameters are trainable. This makes local RL fine-tuning practical, but the very small gradient norms and nearly flat mean log-probabilities observed in the TSP runs suggest limited exploration.

4.6 Discussion

The experiments support three conclusions. First, the heatmap-plus-decoder design can produce feasible routing solutions, and CADO-style fine-tuning improves the selected validation metric after RL. This is clearest for TSP and CVRP, where the metric is decoded route gap. For MDVRP, the improvement is assignment-level: CADO recovers from its early validation accuracy and finishes close to the best checkpoint while maintaining zero capacity violations.

Second, supervised denoising loss is not a reliable proxy for decoded routing quality. In both CVRP and MDVRP, training loss continues to decrease while validation gap or assignment accuracy worsens. This empirically matches the objective-mismatch argument motivating CADO: the network can become better at predicting labels without becoming better after the non-differentiable decoder.

Third, CADO fine-tuning introduces its own stability challenges. The CVRP CADO run

improves substantially at epoch 470 but collapses by epoch 1000. The TSP runs show smaller but noisy improvements. Therefore, the current implementation should be viewed as a successful proof-of-concept for cost-aware fine-tuning rather than a final state-of-the-art solver. Future experiments need larger validation subsets, early stopping, stronger exploration regularization, and full MDVRP route-cost evaluation.

제 5 장 Conclusion

5.1 Summary

This thesis investigated diffusion-based neural solvers for combinatorial optimization, with a focus on routing problems whose feasible solutions must satisfy hard structural constraints. The work builds on graph-based diffusion solvers for combinatorial optimization [12]. The central motivation was the mismatch between supervised heatmap learning and the final decoded solution quality. A diffusion model may predict labels accurately under a surrogate loss, but the final output is produced only after a non-differentiable decoder enforces problem constraints.

To study this issue, the thesis implemented and evaluated a DIFUSCO-style supervised diffusion solver [12], together with CADO-style reinforcement learning fine-tuning [16] for TSP and CVRP. The implementation also includes an assignment-level MDVRP extension. TSP and CVRP use edge heatmaps followed by greedy decoding and 2-opt refinement [3], while MDVRP predicts customer–depot assignments as a first step toward full multi-depot route generation.

The experiments show that CADO fine-tuning can improve the best validation checkpoint over supervised training. On TSP-50, the supervised baseline achieved a 4.46% validation gap, and the 1000-epoch CADO run improved the reported gap to 3.77%. On CVRP-50, supervised training with 2-opt achieved a best gap of 25.96%, while CADO improved the best checkpoint to 20.24%. These results support the CADO motivation that directly optimizing decoded cost can help align diffusion heatmaps with routing objectives [16].

At the same time, the experiments also reveal instability. CVRP CADO collapsed late in training, ending much worse than its best checkpoint. The MDVRP supervised assignment model showed decreasing training loss but deteriorating validation assignment accuracy. These findings reinforce the main thesis claim: for constrained routing problems, the decoder and objective must be treated as central parts of the learning system rather than as afterthoughts.

5.2 Limitations

This work has several limitations. First, the achieved gaps are still far from classical solver quality, especially for CVRP. The current heatmap decoder produces feasible CVRP solutions, but the best CVRP gap remains around 20%, indicating that the learned heatmap and the greedy decoder are not yet strong enough for competitive routing performance.

Second, the validation protocol is limited by subset size. Some CADO experiments use only 100–200 validation instances, so individual best checkpoints may include sampling noise. The reported trends are useful for development, but publication-quality numbers should be computed on larger fixed test sets.

Third, the MDVRP implementation does not yet solve the full multi-depot routing problem. It predicts depot assignments and checks aggregate capacity, but route construction inside each depot is not integrated into the main evaluation loop. Therefore, MDVRP results should be interpreted as assignment-level diagnostics rather than full MDVRP performance.

Finally, CADO fine-tuning is sensitive to reward design, learning rate, training length, and checkpoint selection. The CVRP run demonstrates that a promising mid-training checkpoint can be followed by severe degradation. This makes early stopping and more robust reward normalization necessary for reliable use.

5.3 Future Work

Future work should first strengthen the evaluation protocol. The best TSP and CVRP CADO checkpoints should be re-evaluated on larger fixed validation and test subsets, and all reported comparisons should use identical data splits. Validation-based early stopping should be incorporated directly into the training procedure.

Second, the CVRP decoder and reward should be improved. The current greedy decoder with per-route 2-opt is feasible but leaves substantial gap. Better route construction, stronger local search, or integration with route-level classical heuristics such as LKH-style VRP heuristics [5] may reduce the gap before and after CADO fine-tuning.

Third, MDVRP should be extended from assignment prediction to full route generation. One natural direction is a two-stage pipeline: first predict depot assignments, then solve or learn a CVRP subproblem for each depot. A more ambitious direction is a unified heatmap over both assignment and routing decisions with a decoder that enforces depot, capacity, and route constraints jointly.

Finally, CADO training should be made more stable. Promising directions include entropy regularization, PPO-style clipped updates [11], adaptive reward normalization, shorter schedules with early stopping, and reward functions that include the same local-search operations used at evaluation time. These changes would better align the training objective with the final decoded solutions while reducing the instability observed in long CVRP fine-tuning runs.

제 A 장 Implementation Details

A.1 Shared Configuration

All experiments use Euclidean routing instances with solver-generated labels. Unless otherwise stated, the dataset is split into 90% training and 10% validation with random seed 42. The categorical diffusion process uses $T = 1000$ training diffusion steps with a linear beta schedule from 10^{-4} to 0.02. Validation and CADO rollouts use a cosine subsampling schedule over the diffusion chain.

Table A.1: Shared data and diffusion settings.

Problem	Dataset file	Instance size	Label source	Diffusion setting
TSP-50	`tsp50_128000_concorde.txt`	50 cities	Concorde	categorical, $T = 1000$
CVRP-50	`cvrp50-50_128000_pyvrp.txt`	50 customers, 1 depot	PyVRP	categorical, $T = 1000$
MDVRP-50	`mdvrp10-10x5-5_64000_pyvrp.txt`	50 customers, 2–5 depots	PyVRP	categorical, $T = 1000$

A.2 Supervised DIFUSCO Runs

The supervised experiments train the DIFUSCO-style denoising model with cross-entropy on solver-generated binary labels. AdamW is used with cosine learning-rate decay, weight decay 10^{-4}, dropout 0, and gradient clipping at norm 1.0. The TSP run uses the small model because it was trained locally; the CVRP run uses the paper-size model on A100; and the MDVRP run uses the small model for the assignment-level extension.

Table A.2: Supervised training configurations used for the reported baselines.

Run	Model	Platform	Epochs	Batch	LR	Eval subset	Validation decoding
TSP-50 SL	$h = 128, L = 6$	Apple M4 Pro MPS	50	16	2×10^{-4}	500	greedy + 2-opt, $M_{\text{eval}} = 50$
CVRP-50 SL	$h = 128, L = 6$	VESSL A100	50	64	2×10^{-4}	500	greedy + per-route 2-opt, $M_{\text{eval}} = 50$
MDVRP-50 SL	$h = 128, L = 6$	VESSL A100	50	64	2×10^{-4}	500	assignment decode, $M_{\text{eval}} = 50$

Table A.3: Reported supervised validation outcomes.

Run	Best validation metric	Final validation metric	Wall-clock	Notes
TSP-50 SL	4.46% gap	4.46% gap	17 h 47 min	per-epoch validation curve not recorded due to logging issue
CVRP-50 SL	25.96% gap	35.51% gap	2 h 9 min	best at epoch 10 with 2-opt validation
MDVRP-50 SL	81.89% accuracy	55.75% accuracy	57 min 44 s	assignment-level metric; 0% capacity violation

A.3 CADO Fine-Tuning Runs

CADO fine-tuning starts from the corresponding supervised checkpoint. All reported CADO runs use REINFORCE, AdamW with learning rate 10^{-4}, no weight decay, gradient clipping at norm 1.0, LoRA rank 2, and one fully trainable final graph layer plus the output head. Each epoch uses 128 sampled training instances, accumulated into batches of 32, giving four optimizer updates per epoch. Unless otherwise stated, the rollout uses $M_{\text{train}} = 5$ and a cosine schedule.

Table A.4: CADO fine-tuning configurations used for the reported experiments.

Run	Start checkpoint	Reward	Epochs	Updates	M_{eval}	Eval subset	Platform	Notes
TSP-50 CADO	TSP-50 SL	SR	1000	4000	50	200	Apple M4 Pro MPS	reported TSP CADO run
CVRP-50 CADO	CVRP-50 SL	LCR	1000	4000	25	100	VESSL A100	best checkpoint at epoch 470
MDVRP-50 CADO	MDVRP-50 SL	SR	500	2000	25	100	VESSL A100	assignment-level fine-tuning

Table A.5: Reported CADO validation outcomes.

Run	Best validation metric	Final validation metric	Wall-clock	Main interpretation
TSP-50 CADO	3.77% gap	3.77% gap	about 6–7 h	1000-epoch run improves over 4.46% SL baseline
CVRP-50 CADO	20.24% gap	47.16% gap	7 h 1 min	best checkpoint improves, final checkpoint collapses
MDVRP-50 CADO	80.86% accuracy	80.48% accuracy	1 h 49 min	recovers assignment accuracy with 0% capacity violation

A.4 Evaluation Protocol

TSP and CVRP use the relative gap

$$\text{Gap}(\%) = \frac{L_{\text{pred}} - L_{\text{gt}}}{L_{\text{gt}}} \times 100.$$

For TSP, L_{pred} is computed after greedy heatmap decoding and 2-opt. For CVRP, the supervised baseline reported in the main table uses greedy decoding with per-route 2-opt, whereas the CADO run is evaluated with the implemented CADO CVRP decoder. CVRP validation also records over-capacity route violations; all reported CVRP validation rows have 0% over-capacity rate. MDVRP reports assignment accuracy and capacity-violation rate rather than route-cost

gap, because full route construction within each depot is not part of the current MDVRP experiment.

A.5 Configuration Files and Logs

The Hydra configuration files used by the code are stored under `configs/`. The main entry-point configuration files are:

- supervised: the three `difusco_*_config.yaml` files;
- CADO: the three `cado_*_config.yaml` files.

The exact run values reported above are taken from the experiment logs under `logs/`, because some runs override the YAML defaults from the command line or from the remote execution environment.

참고 문헌

[1] David L. Applegate, Robert E. Bixby, Vasek Chvátal, and William J. Cook. The traveling salesman problem: A computational study. In *The Traveling Salesman Problem*. Princeton University Press, 2011.

[2] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

[3] Georges A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958.

[4] Gurobi Optimization, LLC. Gurobi optimizer reference manual. https://www.gurobi.com, 2023.

[5] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde University*, 12:966–980, 2017.

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[7] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[8] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[10] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Courier Corporation, 1998.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

[12] Zhiqing Sun and Yiming Yang. Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems*, 36:3706–3731, 2023.

[13] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[14] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[15] Niels A. Wouda, Leon Lan, and Wouter Kool. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36(4):943–955, 2024. doi: 10.1287/ijoc.2023.0055.

[16] D. Yoon, H. Song, K. Lee, and W. Lim. Cado: Cost-aware diffusion solvers for combinatorial optimization through rl fine-tuning. In *ICML 2024 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2024.